

Knowing the complexity in competitive programming

Last Updated : 11 Jul, 2025



Prerequisite: [Time Complexity Analysis](#)

Generally, while doing competitive programming problems on various sites, the most difficult task faced is writing the code under desired complexity otherwise the program will get a TLE (**Time Limit Exceeded**).

A naive solution is almost never accepted. So how to know, what complexity is acceptable?

The answer to this question is directly related to the **number of operations that are allowed to perform within a second**. Most of the sites these days allow 10^8 operations per second, only a few sites still allow 10^7 operations. After figuring out the number of operations that can be performed, search for the right complexity by looking at the constraints given in the problem.

Example: Given an array $A[]$ and a number x , check for a pair in $A[]$ with the sum as x , where N is:

- 1) $1 \leq N \leq 10^3$
- 2) $1 \leq N \leq 10^5$
- 3) $1 \leq N \leq 10^8$

- **For Case 1:** A naive solution that is using two for-loops works as it gives us a complexity of $O(N^2)$, which even in the worst case will perform 10^6 operations which are well under 10^8 . Of course, $O(N)$ and $O(N \log N)$ is also acceptable in this case.
- **For Case 2:** We have to think of a better solution than $O(N^2)$, as in worst case, it will perform 10^{10} operations as N is 10^5 . So complexity acceptable for this case is either $O(N \log N)$ which is approximately 10^6 ($10^5 * \sim 10$) operations well under 10^8 or $O(N)$.
- **For Case 3:** Even $O(N \log N)$ gives us TLE as it performs $\sim 10^9$ operations which are over 10^8 . So, the only solution which is acceptable is $O(N)$ which in worst

case will perform 10^8 operations. The code for the given problem can be found on : <https://www.geeksforgeeks.org/dsa/check-if-pair-with-given-sum-exists-in-array/>

Estimating efficiency:

Given: The time limit for a problem is 1 second.

The input size for the problem is 10^5 .

And, if the Time complexity is $O(n^2)$

So, the Algorithm will perform about $(10^5)^2 = 10^{10}$ number of operation

This should take at least 10 seconds, The algorithm seems to be too slow for solving the problem.

How to Determine the solution of a problem by looking at its constraints?

Constraints	Worst time complexity	Algorithmic solution	Examples
$n \leq 12$	$O(n!)$	Recursion & Backtracking	Permutation 1....n
$n \leq 25$	$O(2^n)$	Recursion , Backtracking & Bit Manipulation	All subsets of an array of size n
$n \leq 100$	$O(n^4)$	Dynamic Programming	4Sum
$n \leq 500$	$O(n^3)$	Dynamic Programming	All triangles with side length less

Constraints	Worst time complexity	Algorithmic solution	Examples
			than n
$n \leq 10^4$	$O(n^2)$	Dynamic Programming, Graphs , Trees	Bubble Sort (Slow comparison-based sorting)
$n \leq 10^6$	$O(n \log n)$	Sorting , Binary Search , Divide and Conquer	Merge Sort (Fast comparison-based sorting)
$n \leq 10^8$	$O(n)$	Mathematical , Greedy	Min and max of element
$n > 10^8$	$O(\log n)$ or $O(1)$	Mathematical, Greedy	Binary Search

So, in our previous example given input size $n=10^5$, it is probably expected that the time complexity of the algorithm is $O(n)$ or $O(n \log n)$.

That is how this can help us to design the algorithm easier because it rules out approaches that would yield an algorithm with a worse time complexity.

*It is important to **remember** that time complexity is only an estimate of efficiency because it hides the constant factors. For example, an algorithm that runs in $O(n)$ time may perform $n/2$ or $2n$ operations.*

Let's discuss the classic problem for a better understanding of **time complexity**
Estimating efficiency,

Given an array of n numbers, our task is to calculate the maximum subarray sum. The largest possible sum of a sequence of consecutive values in the array.

-2	-3	4	-1	-2	1	5	-3
----	----	---	----	----	---	---	----

$$4 + (-1) + (-2) + 1 + 5 = 7$$

Maximum contiguous Array sum is 7

The largest possible sum of a sequence of consecutive values in the array.

This problem has a straightforward $O(n^3)$ solution however, by designing a better algorithm, it is possible to solve the problem in $O(n^2)$ time and even in $O(n)$ time.

Method number 1:-

In this method, we go through all possible subarrays, calculate the sum of values in each subarray and maintain the maximum sum.

```
int contiguous_array_sum(int A[ ], int n)
{
    int answer=0;
    for(int i=0; i<n; i++)
    {
        for(int j=i; j<n; j++)
        {
            int sum=0;
            for(int k=i; k<=j; k++)
            {
                sum += A[k];
            }
            answer=max(sum,answer);
        }
    }
    return answer;
}
```

Algorithm 1 with time complexity $O(n^3)$

The variables *i* and *j* fix the first and last index of the subarray, and the sum of values is calculated to the variable *sum*. The variable *answer* contains the maximum sum found during the search.

Time complexity - $O(n^3)$, because it consists of three nested loops.

Method number 2:-

It is really easy to make method number 1 more efficient by removing one loop

from it. This is possible by calculating the sum at the same time when the right end of the subarray moves.

```
int contiguous_array_sum(int A[ ], int n)
{
    int answer=0;
    for(int i=0; i<n; i++)
    {
        int sum=0;
        for(int j=i; j<n; j++)
        {
            sum+=A[ j ];
            answer=max(sum,answer);
        }
    }
    return answer;
}
```

Algorithm 2 with time complexity $O(n^2)$

Time complexity - $O(n^2)$, because it consists of two nested loops.

Method number 3:-

Surprisingly, it is possible to solve the problem in **$O(n)$** time, which means that just one loop is enough. The idea is to calculate, for each array position, the maximum sum of a subarray that ends at that position. After this, the answer to the problem is the maximum of those sums.

The subproblem of finding the maximum-sum subarray that ends at position **i** th. There are two possibilities:

1. The subarray only contains the element at position **i** th.
2. The subarray consists of a subarray that ends at position **$i - 1$** , followed by the element at position **i** .

As we want to find a subarray with the maximum sum, the subarray that ends at position **$i - 1$** should also have the maximum sum. Thus, we can solve the problem efficiently by calculating the maximum subarray sum for each ending position from left to right.

```

int contiguous_array_sum(int A[ ], int n)
{
    int answer=0,sum=0;
    for(int i=0; i<n; i++)
    {
        sum=max(A[i],sum+A[i]);
        answer=max(sum,answer);
    }
    return answer;
}

```

Algorithm 3 with time complexity $O(n)$

Time complexity - $O(n)$, because it consists of one loop, This is also the best possible time complexity, because any algorithm for the problem has to examine all array elements at least once.

Let's compare all these three methods against all possible given input sizes and see their **execution time**:

Array size (n)	Method number 1	Method number 2	Method number 3
10^2	0.0 s	0.0 s	0.0 s
10^3	0.1 s	0.0 s	0.0 s
10^4	> 10.0 s	0.1 s	0.0 s
10^5	> 10.0 s	5.0 s	0.0 s
10^6	> 10.0 s	> 10.0 s	0.0 s
10^7	> 10.0 s	> 10.0 s	0.0 s

compare different methods/Algorithm with different Array sizes

It is interesting to see how efficient algorithms are in practice. The above table shows the running times of the above algorithms/methods for different values of n . The comparison shows that all algorithms are efficient when the input size is small, but larger inputs bring out remarkable differences in the running times of the algorithms.

Method 1 becomes slow when $n = 10^4$, and **method 2** starts to become slow when $n = 10^5$. Only **method 3** is able to process even the largest inputs instantly.

 Comment

More info 

Advertise with us

Explore

DSA Fundamentals



Data Structures



Algorithms



Advanced



Interview Preparation



Practice Problem



Do Not Sell or Share My Personal Information